MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Agents in the Physical World

Wei-Chun Huang, Yun-De Liu, Abhi Patel

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

ASPIRE: Agentic /Skills Discovery for Robotics

Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi “Jim” Fan, Guanzhi Wang

Background

Traditional	VLA / Imitation	Code-as-Policy	ASPIRE
Hand-written robot control scripts	Behavior learned from demonstrations	LLM writes executable robot programs	Code, execution traces, repairs and transferable skills

From one-off task execution to cumulative debugging experience

Problems with Code-as-policy

1. Coarse feedback

“Task failed” does not say which perception, planning or grasp primitive broke.

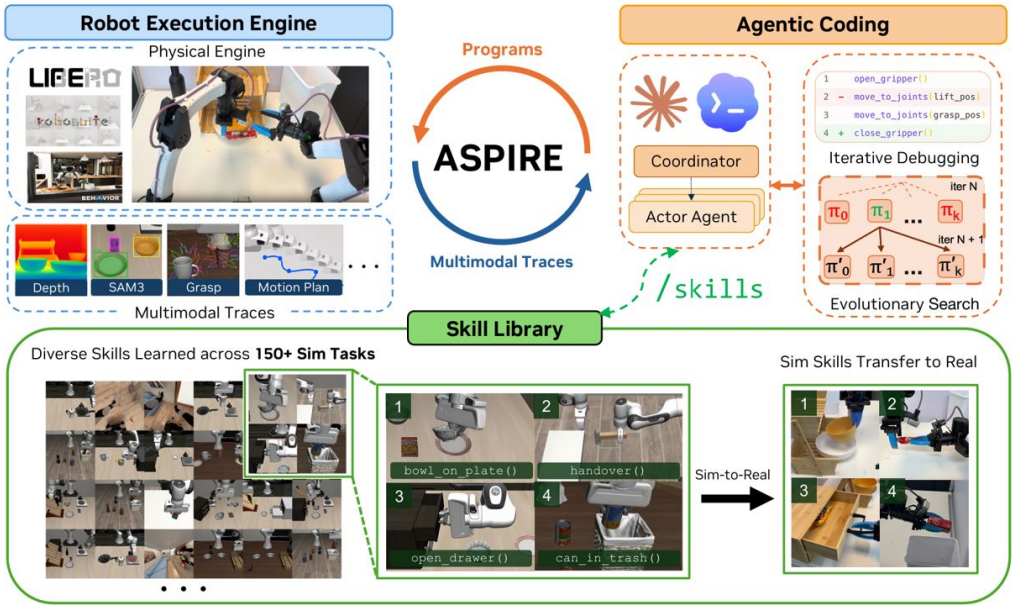
---

2. No durable learning

Successful fixes are discarded instead of reused on later tasks.

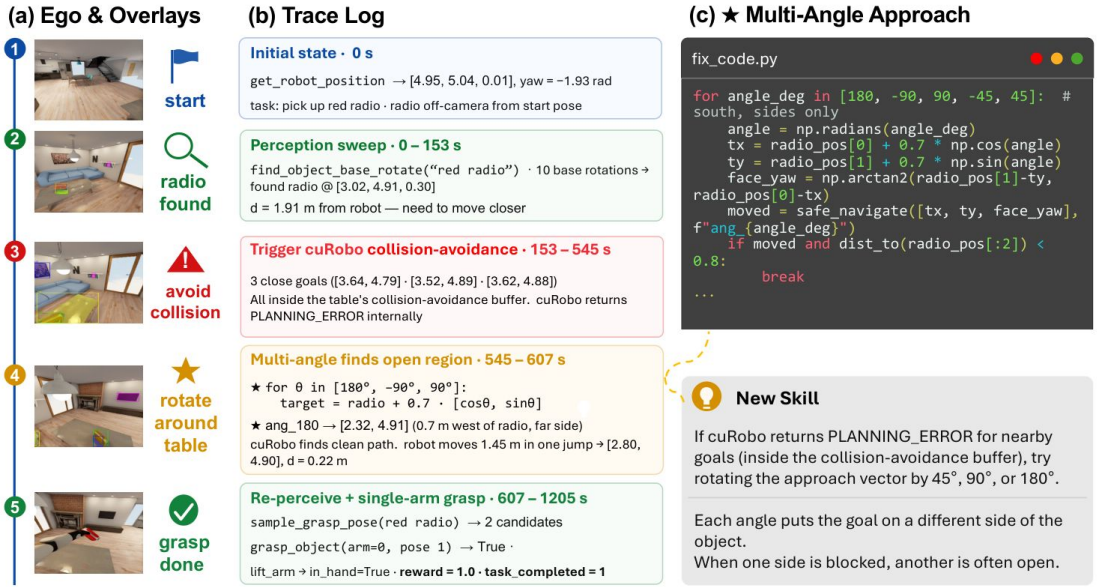
ASPIRE’s answer Fine-grained traces + persistent skills + diverse program search

The System



- Robot Execution Engine
  - Records program execution and multimodal, per-primitive traces.
- Skill Library
  - Stores validated repairs as reusable skills.
- Evolutionary Search
  - Generates diverse candidate programs and retains useful solutions.

Robot Execution Engine



Trace-guided debugging: planning error → multi-angle repair → validated new skill

Technical Details: Skill Library

- **Heterogeneous Repair Distillation:** Stores validated, agent-discovered repairs as modular, transferable in-context guidance rather than fixed human-written code primitives.
- **Structured Entry Schema:** Captures the observed failure signature, when-to-apply activation guard, verified repair strategy, and representative code snippets.
- **Diverse Skill Categories:** Encompasses localization/disambiguation, perception prompting, grasping constraints, navigation recovery (e.g., multi-angle approaches), low-level motion primitives, scene reasoning, and debugging workflows.
- **Cross-Task & Cross-Embodiment Transfer:** Functions as in-context prompts for future actor agents to enable zero-shot transfer on unseen long-horizon tasks and cross-embodiment guidance for real-robot manipulation.

Technical Details: Evolutionary Search

1 Generate candidates

Propose K alternative task sequences and control programs, rather than patching one failed path.

---

2 Execute & select

Run candidates; inspect primitive traces; retain stronger programs and unresolved failure evidence.

---

3 Evolve & stop

Generate offspring from survivors, the skill library and a task-analysis ledger; stop on validated success or budget.

Technical Details: Coordinator–Actor System

Coordinator

Dispatches tasks to parallel actors.

Audits successful repairs.

Serializes writes to the shared skill library.

Reruns low-scoring tasks with the final library.

Actor

Writes code-as-policy programs.

Calls robot execution APIs.

Inspects multimodal traces and patches the failure.

Validates fixes by re-execution.

Results: LIBERO-Pro task suites

Suite	CaP-Agent0	ASPIRE	Gain (pp)
Object	20.0%	96.5%	+76.5
Goal	21.5%	63.0%	+41.5
Spatial	13.0%	55.5%	+42.5
Overall	18.0%	72.0%	+54.0

Object improves by 76.5 pp, rounded to 77 pp in the paper

Results: Gains extend to bimanual and mobile tasks

Benchmark / task	CaP-Agent0	ASPIRE	Trials / task
Robosuite handover	20%	92%	100
BEHAVIOR soda can	72%	88%	25
BEHAVIOR radio	56%	88%	25
Long zero-shot, N=90	3.8%	30.5%	50

Zero-shot: 30.5% with N=90 source tasks, without test-time retries

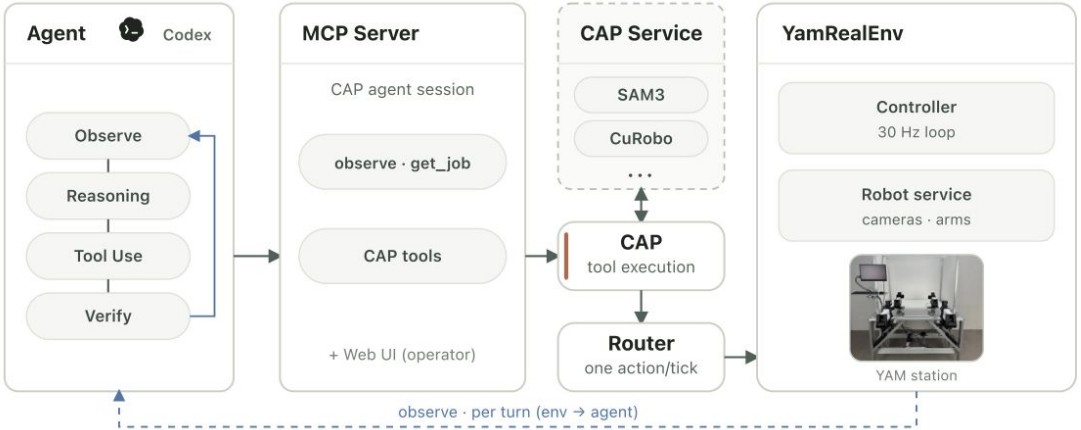
Results: Simulation skills reduce real-robot debugging cost

Task	Tokens w/o	Tokens with	Success w/o	Success with
Bowl on plate	8.65M	5.11M	20/20	20/20
Lift soda can	61.94M	6.58M	13/20	19/20
Open/push drawer	334.92M	81.67M	0/20	11/20

Soda-can debugging uses 9.4× fewer total tokens with skill guidance

Transfer uses textual skill guidance. The real robot still needs execution-based adaptation.

A persistent runtime connects reasoning to control



Agent sessions and robot execution have separate lifetimes

Validation can select a weaker test program

Task / perturbation	Debugged	Evo candidate	Selected
In top drawer / Position	22%	10%	10%
Between plate + ramekin / Task	8%	84%	8%

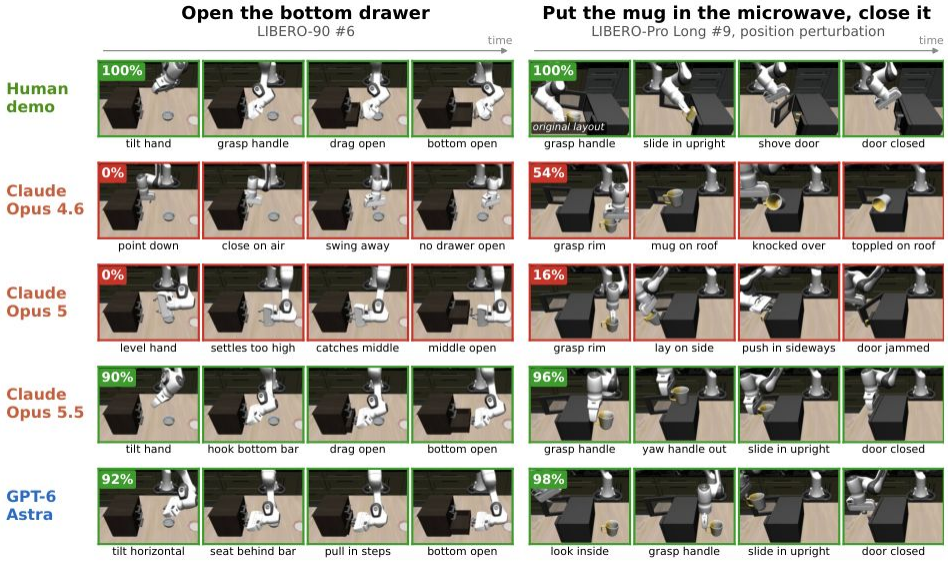
Stronger candidate search also needs reliable selection

Most token traffic comes from cache reads

Metric	Opus 5.5	GPT-6 Astra
Cache-read / total tokens	1,493.5M / 1,538.3M	505.4M / 522.1M
Cache-read share	97.1%	96.8%
LLM calls / simulator rollouts	7,896 / 10,555	5,950 / 7,905
Cost / actor time per task	$10.8 / 46 min	$12.9 / 27 min

Systems opportunity: context reuse and cache-aware scheduling

Grasp choice constrains the rest of the task



Early actions must preserve the feasibility of later subgoals

Search still helps after stronger models find a strategy

Case	Opus 5.5	+ Search	Astra	+ Search
Book under shelf	68%	100%	22%	96%
Push plate to stove	84%	94%	62%	92%
Open bottom drawer	90%	90%	92%	100%

Strategy discovery and program reliability are complementary

Strengths & Limitations

Strengths	Limitations
Primitive traces localize failures	Debugging and search require many rollouts
Reusable skills carry experience across tasks	Results depend on the underlying LLM
Selected skills transfer across embodiments	Admission checks do not prevent stale skills
Held-out seeds test repaired programs	Small real-robot study, with human resets

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

TimelyLLM: Time-sensitive LLM Serving System for Physical-I/O Limited Agents

Neiwen Ling, Guojun Chen, Anurag Khandelwal, Lin Zhong

Physical-I/O agents consume output slowly

Robots execute actions. Voice assistants play speech.
Both consume generated output sequentially.

LLM generation

~100 tokens/s

GPT-4o example in the paper

Physical consumption

Seconds per action

Drone: 2.53 s to move 1 m
Speech: about 150 words/min

Physical execution time creates scheduling slack

Three gaps in conventional serving

01 Over-generation

Generate later steps before the agent needs them.

02 Unused execution slack

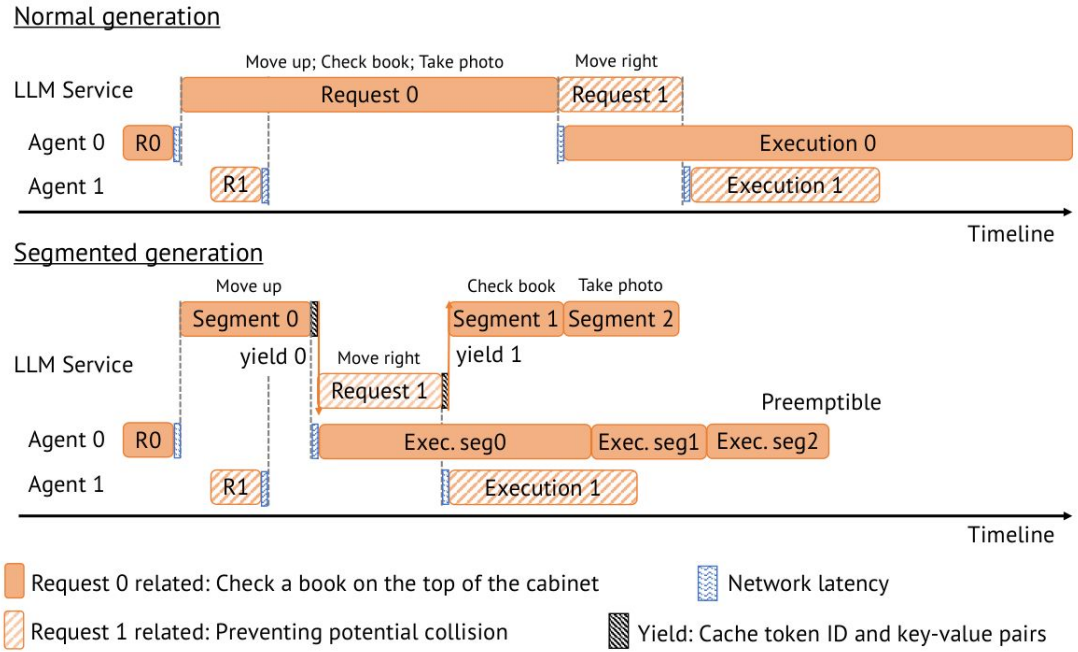
Continue A's plan while A acts; urgent B waits for compute.

03 Urgency-blind scheduling

FCFS or static priority misses changing deadlines.

Generate each next segment when it is needed

Turn physical execution time into slack



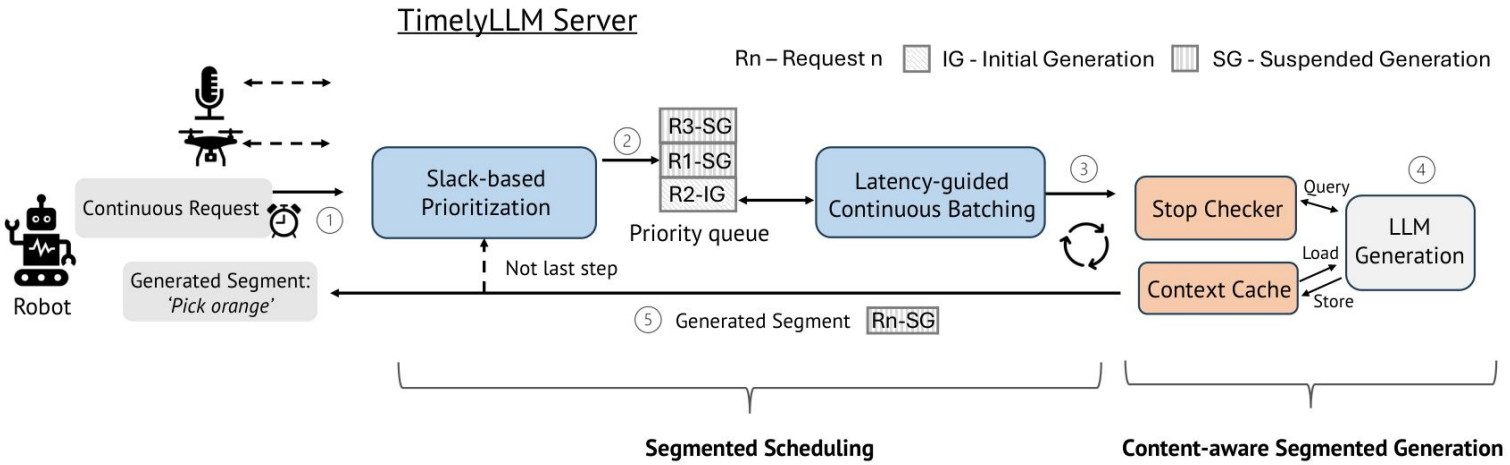
1 Generate one action

- Agent 0 starts moving

2 Yield to urgent work

- Serve Agent 1 while Agent 0 executes
- Resume before Agent 0 needs its next action

TimelyLLM system overview



Segment

Stop at an executable boundary

Prioritize

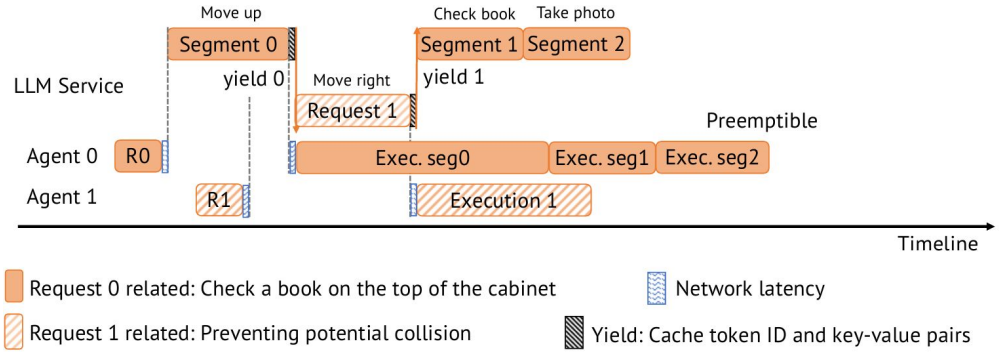
Use slack and request urgency

Batch

Admit work that fits timing limits

Content-Aware Segmentation

Split the output into meaningful, executable segments.



A segment is ready only when it passes two checks

- **Structural Completeness**
- **Time Compliance**

Two Things go into the context cache:

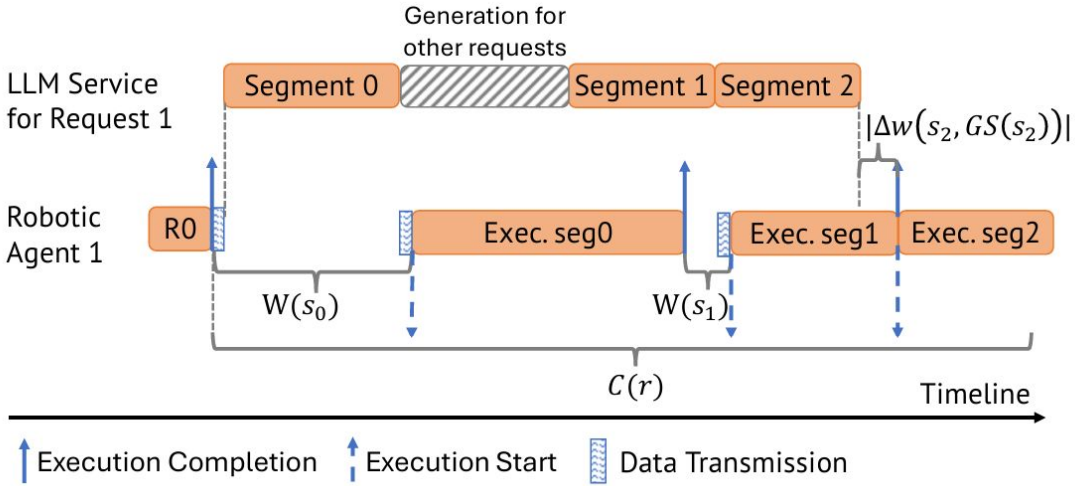
KV Cache

Output Token IDs

Overhead across different tasks:

Trace ID	1	2	3	4	5	6	7	8
Overhead (ms)	13.7	24.1	19.9	19.9	16.0	13.1	14.4	14.8
Overhead (%)	3.0	2.6	5.6	2.8	3.2	3.7	4.1	4.2

Slack-Based Scheduling



$$\Delta(s_k^i, t) = t + G + C - EE(s_{k-1}^i)$$

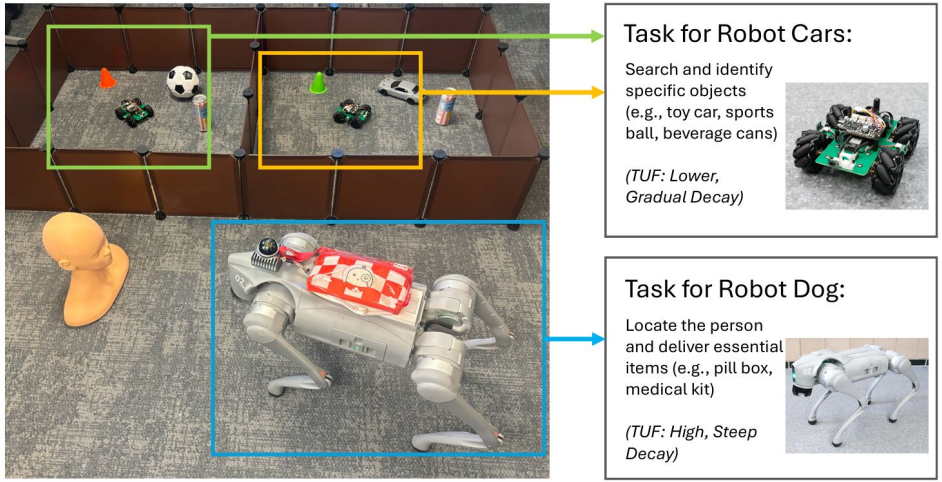
Evaluations

- Methods
  - Main Benchmark: LRTrace
  - Real World Evaluations
    - Multi-Robot Testbed
    - Speech Agents

Categories	Trace ID	Task Description
Normal request for drone	1	Look for the cat
	2	Check for keys or wallets
	3	Check the book on the cabinet
	4	Find a toy under the bed
	5	Find a tool for cutting paper
Urgent request for drone	6	Avoiding human injury
	7	Preventing potential collision
	8	Evading animal pursuit
Complex request for robot arm	9	Object stacking
	10	Desk cleaning
	11	Fruit and vegetable classification
Request for quadruped robot	12	Find the household item
	13	Guide to the door
	14	Bark to deter intruders

- Setup:
  - Models: Llama3-8B-float16.
  - Hardware: in-house edge server, equipped with an NVIDIA RTX 4090 GPU, an AMD Ryzen 9 7950X CPU, and 24 GB of host memory.

Real World Evaluation 1: Robot Agents



Task for Robot Cars:

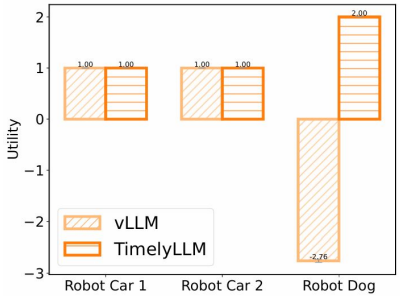
Search and identify specific objects (e.g., toy car, sports ball, beverage cans)

(TUF: Lower, Gradual Decay)

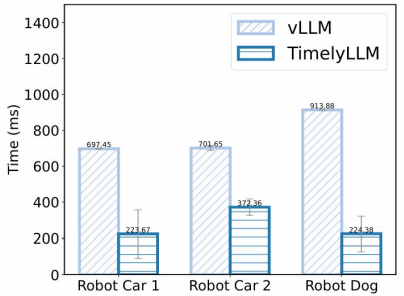
Task for Robot Dog:

Locate the person and deliver essential items (e.g., pill box, medical kit)

(TUF: High, Steep Decay)



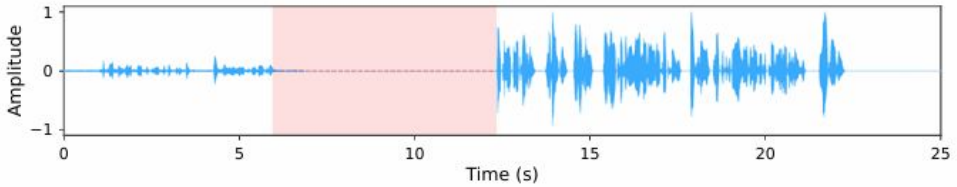
(a) Time Utility



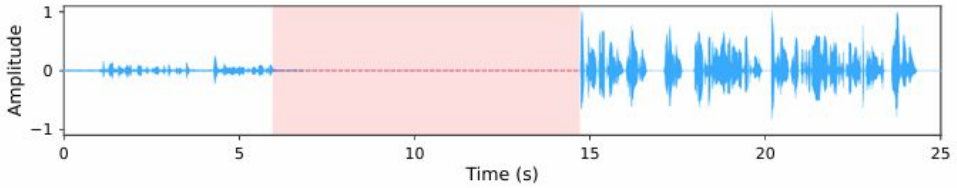
(b) Waiting Time

ERT = 1 s, beta = 1, alpha = -2
ERT = 200 ms, beta = 2, alpha = -6.67

Real World Evaluation 2: Spoken Agents

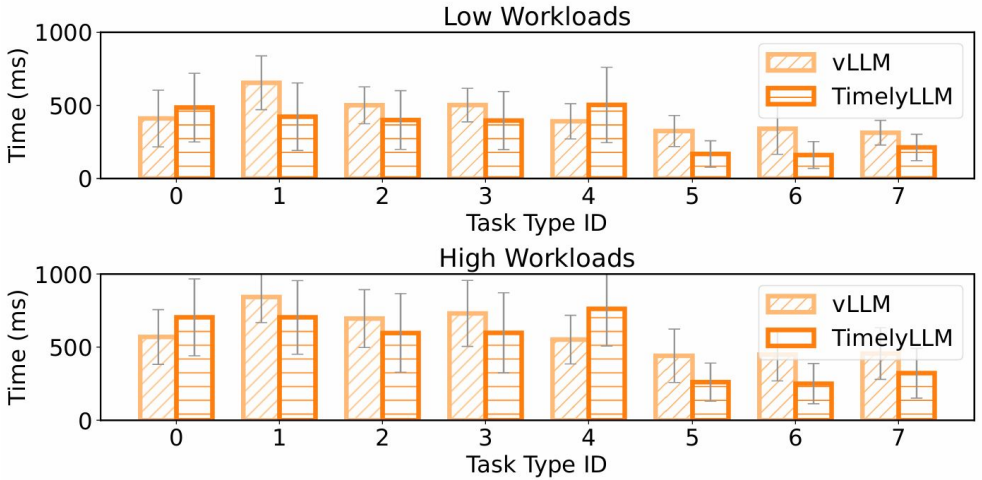


(b) Waveform - TimelyLLM



(c) Waveform - vLLM

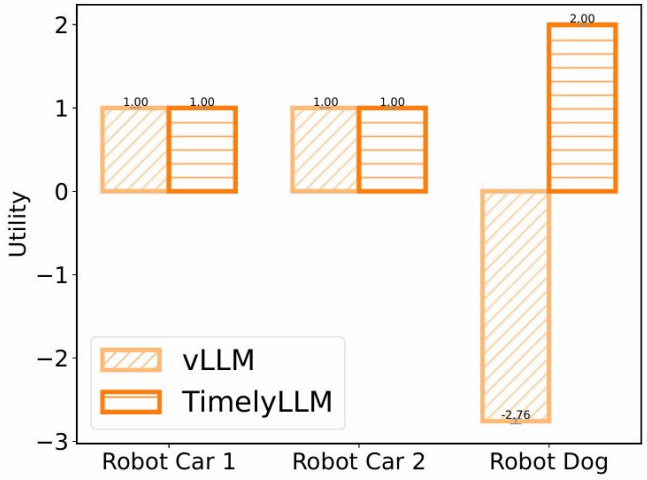
How does Timely LLM perform in time sensitivity?



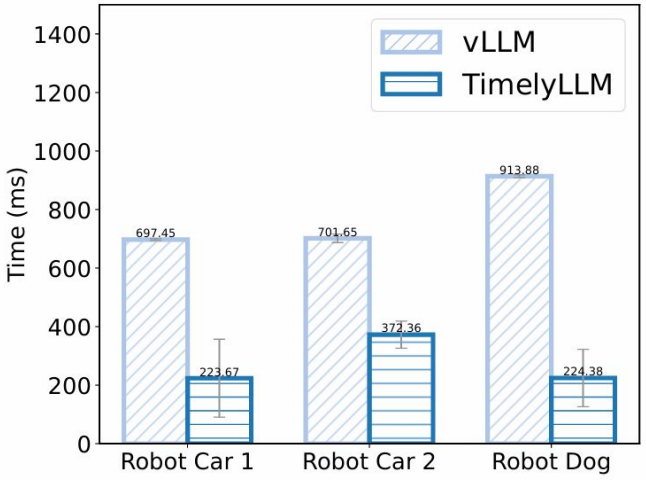
(b) Response time.

WID	Categories	EPS	Max TPE	Trace ID	AN	Time (s)
1	Mixed TUF	0.25	8	1-8	25	260
2	High-Parallel	0.25	16	1-8	42	300
3	Arm - *TypeFly*	0.1	8	9-11	40	900
4	Arm - *FLTRNN*	0.5	8	9-11	81	100
5	Mixed Robots	0.25	12	3-8,12+	47	500

How does TimelyLLM perform in time sensitivity?

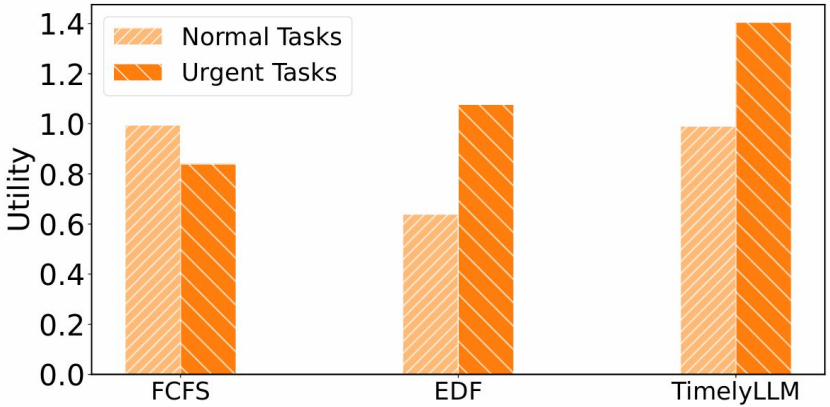


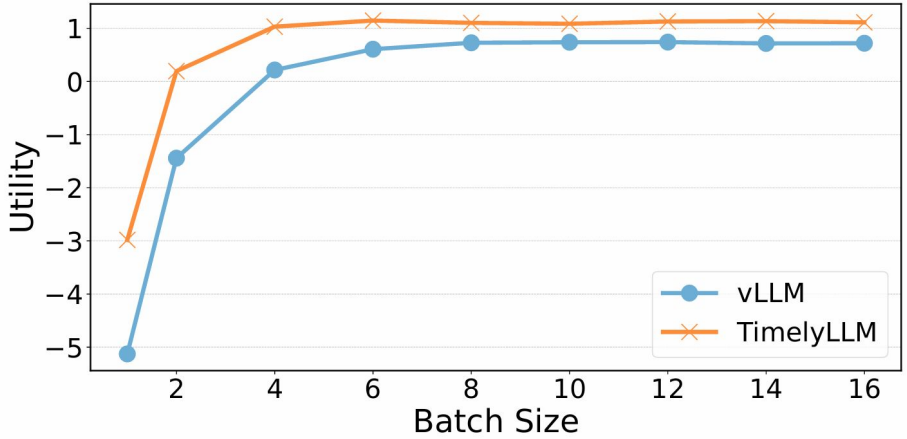
(a) Time Utility



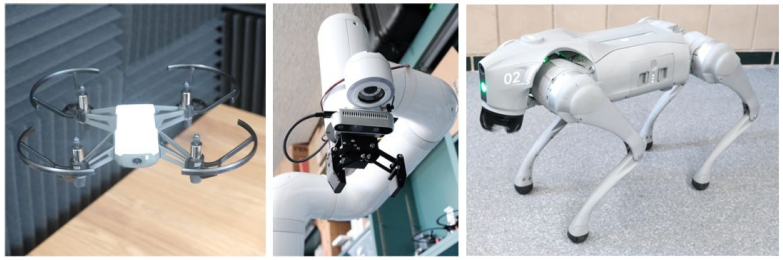
(b) Waiting Time

Which system components enhances TimelyLLM?

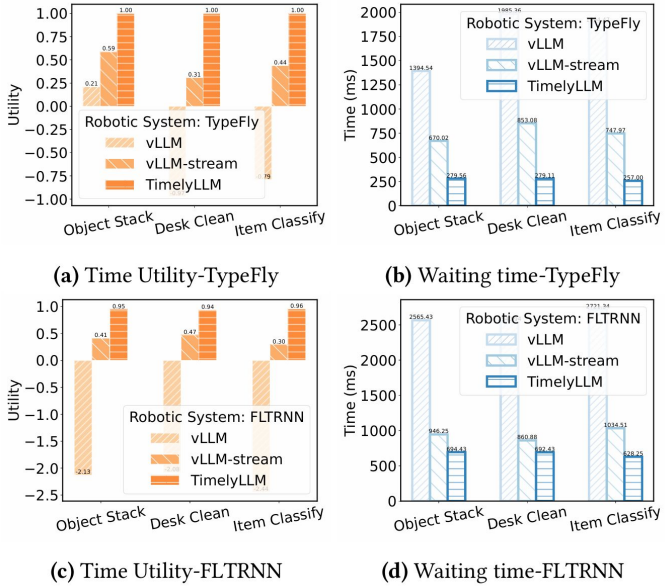




Does TimelyLLM Generalize?



(a) Drone (b) Robot Arm (c) Quadruped Robot



(a) Time Utility-TypeFly (b) Waiting time-TypeFly

(c) Time Utility-FLTRNN (d) Waiting time-FLTRNN

System Overhead

Table 6: *The segmented overhead across eight traces, presented in milliseconds and as a percentage.*

Trace ID	1	2	3	4	5	6	7	8
Overhead (ms)	13.7	24.1	19.9	19.9	16.0	13.1	14.4	14.8
Overhead (%)	3.0	2.6	5.6	2.8	3.2	3.7	4.1	4.2

Table 7: *The scheduling overhead for various sizes of task queues.*

Task Number	1	2	4	6	8
Scheduling Overhead (ms)	0.58	0.83	1.25	1.58	2.03

Strengths & Limitations

Strengths

Clear insight. Robots act slower than LLMs generate, so execution time is slack worth reclaiming.

Low overhead. Segmentation under 6% of generation time; scheduling about 2 ms.

Real systems. Physical robots and a voice pipeline, built on vLLM, with code and dataset released.

Theory plus ablations. Competitive-ratio bound, and FCFS/EDF tests isolate the scheduler.

Limitations

Needs slack. Smaller gains for tight feedback loops like ReAct, as the authors note.

Hand-set parameters. Priorities depend on chosen α, β and profiled action times.

Small scale. One RTX 4090, 7–8B models, up to 16 tasks; multi-GPU only discussed.

Narrow comparison. Self-built benchmark; baselines are mostly vLLM variants.



Q & A